Bitonic Sort: A Parallel Sorting Network

Nishchay Bansal [RA2411028010003] – Nikunj Purohit [RA2411028010018] – Arnav Rana [RA2411028010041]

SRM Institute of Science And Technology, Kattankulathur – 603203, Chennai, India

Faculty Guide: Dr. Prabhu Chakkaravarthy

ABSTRACT

Bitonic Sort is a parallel sorting algorithm designed for systems that perform multiple comparisons simultaneously. The algorithm works by first constructing bitonic sequences—sequences that increase and then decrease—and then recursively merging them into a fully sorted order. Due to its regular comparison pattern, Bitonic Sort is widely used in parallel processors and hardware sorting networks. Although its theoretical complexity is higher than some sequential algorithms, it performs efficiently in parallel architectures.

PROBLEM STATEMENT

Sorting is a fundamental operation in computer science and is widely used in data processing, searching, and database management. Traditional sequential sorting algorithms such as Quick Sort or Merge Sort perform efficiently on single processors but do not fully exploit the capabilities of parallel hardware. With the rise of parallel architectures such as GPUs and multi-core processors, there is a need for algorithms that can perform multiple comparisons simultaneously. Bitonic Sort addresses this challenge by providing a structured sorting network suitable for parallel execution.

CONCEPT

Bitonic Sort works by recursively dividing an array into two halves. The first half is sorted in ascending order while the second half is sorted in descending order to create a bitonic sequence. A bitonic merge operation then compares and swaps elements at fixed distances to gradually produce a fully sorted sequence.

Steps:

- Divide the array into two halves
- Sort first half in ascending order
- Sort second half in descending order
- Apply Bitonic Merge
- Repeat recursively until sorted

BITONIC SEQUENCE

A bitonic sequence is a sequence of numbers that first increases and then decreases, or vice versa.

Example:

1 4 7 9 6 3 2

Here the sequence increases from 1 to 9 and then decreases to 2.

Bitonic sorting algorithms operate by constructing such sequences and then applying a merge process that gradually produces a fully sorted order.

OBJECTIVE

- Understand the concept of **bitonic sequences** used in parallel sorting
- Study the **divide-and-conquer strategy** used in Bitonic Sort
- Implement Bitonic Sort using **recursive procedures**
- Analyze the **time complexity O(n log² n)** of the algorithm
- Examine how **compare–swap operations** create sorted sequences
- Demonstrate how Bitonic Sort works efficiently in **parallel architectures**
- Explore its role in **GPU and hardware sorting networks**

WORKING PRINCIPLE

The Bitonic Sort algorithm follows three major stages:

Divide

The input array is recursively divided into two halves.

Construct Bitonic Sequence

One half is sorted in ascending order while the other half is sorted in descending order.

Bitonic Merge

Elements at specific distances are compared and swapped.

This process is repeated recursively until the sequence becomes fully sorted.

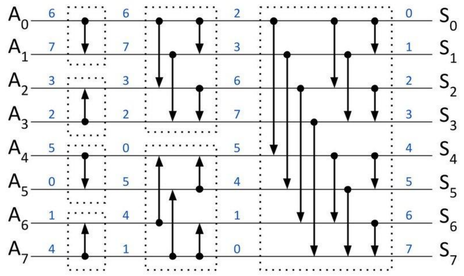
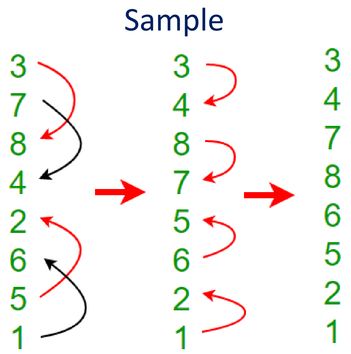


Figure-1:
Bitonic sorting network illustrating parallel compare–swap operations for 8 elements.

PSEUDO CODE

```
Procedure BITONIC_MERGE(A, low, count, direction)

1. if count > 1 then
2.     k ← count / 2

3.     for i ← low to low + k − 1 do
4.         if direction = ASCENDING then
5.             if A[i] > A[i + k] then
6.                 swap(A[i], A[i + k])
7.             end if

8.         else
9.             if A[i] < A[i + k] then
10.                swap(A[i], A[i + k])
11.            end if
12.        end if

13.    end for

14.    BITONIC_MERGE(A, low, k, direction)
15.    BITONIC_MERGE(A, low + k, k, direction)

16. end if
```



ADVANTAGES AND LIMITATIONS

ADVANTAGES

- Highly suitable for parallel processors
- Deterministic comparison pattern
- Efficient for hardware implementation
- Works well in GPU-based architectures

LIMITATIONS

- Higher complexity compared to Quick Sort for sequential systems
- Requires number of elements to be a power of two (or padding)
- Increased comparison operations compared to optimized sequential algorithms

CALCULATION BY MASTER'S THEOREM

Variables

$a = 2 \rightarrow$ number of recursive calls

$b = 2 \rightarrow$ input size divided by 2 each step

$$f(n) = O(n \log n)$$

Recurrence Relation

T(n)=2T(n/2)+O(nlogn)

Compute

n^(log₂ 2) = n

Since

$$f(n) = n \log n = n^{\log_b a} \log n$$

Master Theorem Case 2

$$T(n) = \Theta(n \log^2 n)$$

Final Time Complexity

$$O(n \log^2 n)$$

APPLICATIONS

- GPU sorting algorithms
- Network switching hardware
- Parallel computing systems
- High-performance computing
- Data processing in distributed systems

CONCLUSION

Bitonic Sort is an important algorithm for parallel computing systems due to its predictable structure and suitability for hardware implementation. While it may not outperform algorithms such as Quick Sort in sequential environments, it becomes highly efficient when implemented on GPUs and parallel processors. Its deterministic design makes it valuable in high-performance computing applications.

REFRENCES

Knuth — *The Art of Computer Programming*
Wikipedia
GeeksforGeeks
GitHub

GitHub